Banco de Dados

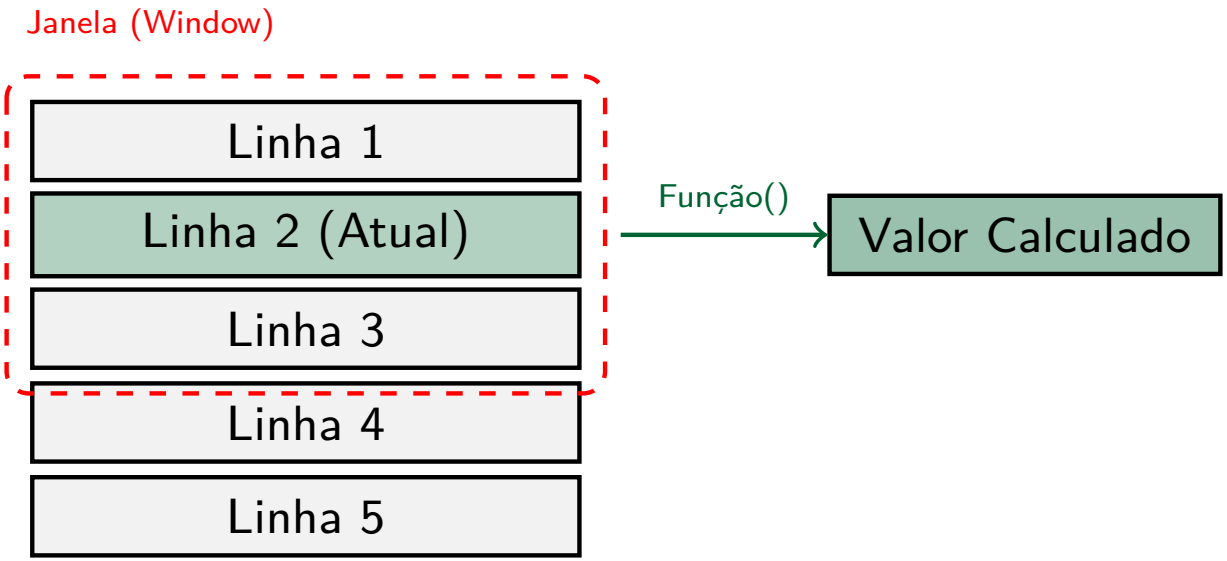
Window Functions, Views e Triggers

Professor: Me. Sergio Souza Novak
Faculdade de Engenharia de Software



O que são Window Functions?

As **Window Functions** em SQL fazem cálculos em um **conjunto de linhas** relacionadas à linha atual, **sem agrupar** ou remover as linhas originais.



- Todas as linhas da consulta original são **preservadas**.
- O resultado é apenas **adicionado** como uma nova coluna.

Quando usar Window Functions?

- **Análises complexas:** Totais acumulados, médias móveis e rankings.
- **Comparações temporais:** Comparar valor atual com o anterior ou seguinte.
- **Otimização de Código:** Substituem *subqueries* e *self-joins* lentos.

Soma Acumulada

10 → **10**
20 → **30**

15 → **45**

Rankings

1º → Ana (10)
2º → João (8)

2º → Bia (8)

Comparações

Atual: R$ 50
Anterior: R$ 40

Crescimento: + **R$ 10**

A Cláusula OVER()

- O uso das Window Functions é habilitado pela palavra-chave **OVER()**.
- Ela determina exatamente qual é a "janela" (ou quadro/conjunto de linhas) que será considerada para o cálculo de cada linha específica da consulta.
- Pode ser usada vazia `OVER()`, o que significa que a "janela" é toda a tabela resultante da consulta.

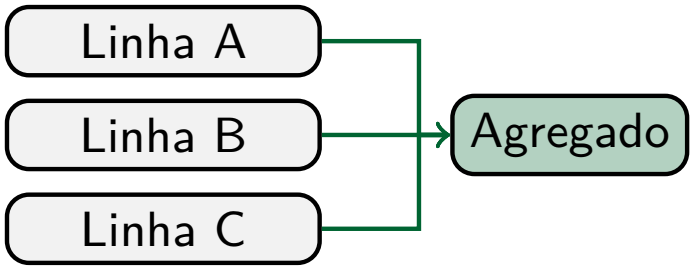
Agregação Comum vs Window Function

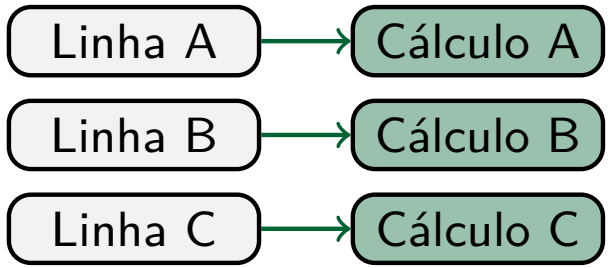
GROUP BY

Retorna apenas **uma linha por grupo**. As linhas originais perdem sua identidade.

Window Function

Mantém **todas as linhas**. O cálculo é apenas anexado como uma nova coluna.



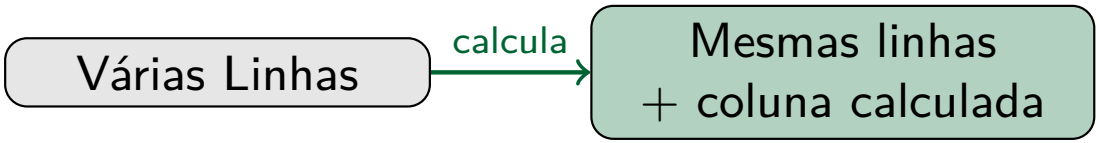


Exemplo Visual: A Ideia Principal

GROUP BY:



WINDOW FUNCTION:



Agregação Comum: Exemplo de Código

Considere uma tabela de **alunos** com ID, Nome, Idade e Curso.
Com uma agregação normal:

```
SELECT curso_id, AVG(idade) AS media_idade
FROM alunos
GROUP BY curso_id;
```

- Nós agrupamos por `curso_id`.
- O resultado não possuirá as informações individuais (como o nome) dos alunos.

Agregação Comum: Resultado

O resultado da consulta anterior nos mostra apenas a média por curso. Perdemos quem são os alunos (Ana, João, etc.).

curso_id	media_idade
1	21
2	25
3	22
4	26
5	20
6	23

Window Function: Exemplo de Código

Como fazer a mesma média, mas preservando as linhas originais?

```
SELECT
    nome,
    idade,
    curso_id,
    AVG(idade) OVER(PARTITION BY curso_id) AS avg_idade
FROM alunos;
```

- Note a ausência do `GROUP BY`.
- O cálculo é feito pela cláusula `OVER`.

Window Function: Resultado

A tabela final mantém todos os alunos, mas adiciona a média correspondente do curso de cada um.

nome	idade	curso_id	avg_idade
Ana	20	1	21
João	22	1	21
Carlos	24	2	25
Maria	26	2	25
Bia	21	3	22
Pedro	23	3	22
Lucas	26	4	26

Anatomia de uma Window Function

A estrutura sintática completa é:

```
FUNCAO_DE_JANELA() OVER(
    [PARTITION BY coluna_agrupamento]
    [ORDER BY coluna_ordenacao]
    [CLAUSULA_DE_FRAME (ROWS ou RANGE)]
)
```

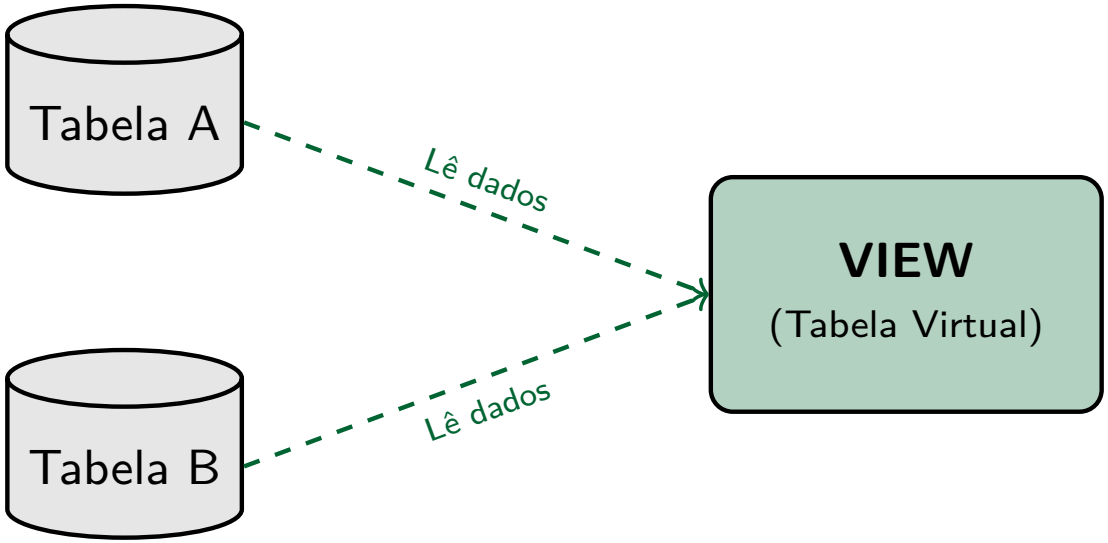
- **FUNCAO**: `SUM`, `AVG`, `RANK`, `LEAD`, etc.
- **PARTITION BY** (opcional): Divide a janela.
- **ORDER BY** (opcional): Ordena dentro da janela.
- **FRAME** (opcional): Restringe ainda mais a janela (ex: apenas as 3 linhas anteriores).

Entendendo o PARTITION BY

- Divide os dados em partições ou blocos lógicos.
- A Window Function é calculada independentemente para cada partição.
- Quando os dados de uma partição acabam, o cálculo é reiniciado para a próxima.
- **Semelhança:** Age de forma análoga ao `GROUP BY`, mas apenas limitando a área de atuação da função, sem esmagar as linhas.
- **Omissão:** Se não especificado, toda a tabela do `SELECT` é considerada uma única partição.

O que é uma View?

Uma **View** (visão) em SQL é uma **tabela virtual** criada a partir de uma consulta (`SELECT`).



Características principais:

- **Não armazena dados fisicamente** no disco.
- Armazena **apenas a consulta SQL** no banco de dados.
- Quando consultada, mostra os dados como se fosse uma tabela normal.

A Ideia Principal

- Uma view funciona como um **“atalho”** para consultas complexas.
- Em vez de escrever uma consulta grande (com múltiplos `JOIN`s, funções de agregação, filtros) repetidas vezes, você a encapsula em uma View.
- Depois, basta consultar a view diretamente, simplificando muito a escrita de novos códigos SQL.

Exemplo Prático: Tabelas de Origem

Suponha que temos duas tabelas relacionadas:

Tabela: alunos

id	nome	curso_id
1	Ana	1
2	João	2
3	Bia	1
4	Pedro	3
5	Lucas	2

Tabela: cursos

id	nome_curso
1	Computação
2	Matemática
3	Física

Criando uma View

Para evitar fazer um `JOIN` toda vez que quisermos ver os alunos e seus cursos, podemos criar uma view:

```
CREATE VIEW vw_alunos_cursos AS
SELECT
    alunos.nome,
    cursos.nome_curso
FROM alunos
JOIN cursos
ON alunos.curso_id = cursos.id;
```

- A palavra-chave é `CREATE VIEW`.
- Convenciona-se usar prefixos como `vw_` ou `v_` para identificar visualmente que o objeto é uma view.

Usando a View e Resultado

Depois da criação, a view pode ser consultada da mesma forma que consultamos uma tabela comum:

```
SELECT * FROM vw_alunos_cursos;
```

Resultado:

nome	nome_curso
Ana	Computação
Bia	Computação
João	Matemática
Lucas	Matemática
Pedro	Física

Vantagens das Views (Parte 1)

Simplificam Consultas

Evitam a repetição de lógicas `SQL` grandes e complexas. Os desenvolvedores e analistas podem acessar os dados sem conhecer a estrutura de `JOIN`s por trás.

Organização

Centraliza a lógica de negócios e regras complexas no banco de dados. Se a lógica mudar, atualiza-se apenas a view, não todo o código da aplicação.

Vantagens das Views (Parte 2)

Segurança e Controle de Acesso

Você pode mostrar apenas algumas colunas e linhas específicas para determinados usuários.

- Exemplo: Uma view que mostra `nome` e `departamento`, mas **esconde** `CPF` e `salário` dos funcionários.

Reutilização

Várias consultas e até diferentes sistemas ou relatórios podem utilizar a mesma view como fonte padronizada de dados.

Tabela vs View

Tabela (Table)	**View (Visão)**
Armazena dados fisicamente	**Não** armazena dados
Ocupa espaço físico no disco	Armazena **apenas a consulta** (SQL)
Contém dados próprios	Exibe dados vindos de **outras tabelas**
É a fonte real das informações	É uma interface/filtro sobre as informações

Alterando uma View

Em muitos bancos de dados (como PostgreSQL e MySQL), podemos alterar a definição de uma view existente usando `CREATE OR REPLACE VIEW`:

```
CREATE OR REPLACE VIEW vw_alunos_cursos AS
SELECT
    alunos.nome,
    cursos.nome_curso,
    alunos.idade -- nova coluna adicionada
FROM alunos
JOIN cursos
ON alunos.curso_id = cursos.id;
```

Alterando uma View no SQLite

No **SQLite**, o comando `OR REPLACE` não é suportado nativamente para views. Normalmente é mais comum apagar a view e recriá-la:

```
-- 1. Apaga a view antiga
DROP VIEW vw_alunos_cursos;

-- 2. Cria a nova view atualizada
CREATE VIEW vw_alunos_cursos AS
SELECT ...;
```

Removendo uma View

Caso a view não seja mais necessária, basta utilizar o comando `DROP VIEW`.

```
DROP VIEW vw_alunos_cursos;
```

- **Importante:** O `DROP VIEW` apaga apenas a definição da view.
- As tabelas originais (`alunos` e `cursos`) e os seus dados continuam intactos e **não** são apagados.

Observação Importante: Atualização Automática

Dinamicidade

Uma view comum **não copia** os dados. Ela sempre mostra os dados atualizados das tabelas originais em tempo real.



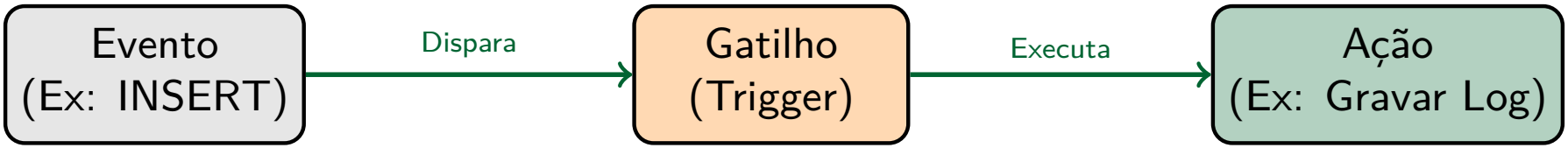
Ou seja: alterou a tabela → a view reflete imediatamente na próxima consulta.

O que são Triggers?

Um **Trigger** (Gatilho) é um tipo especial de rotina (ou procedimento) no banco de dados que é **executado automaticamente** em resposta a certos eventos.

- Ficam armazenados no próprio banco de dados.
- Estão diretamente associados a uma tabela ou view.
- São ativados ("disparados") invisivelmente no plano de fundo.

A Ideia Principal



Ação Reativa

Você não invoca um trigger manualmente com um `CALL` ou `SELECT`. O banco de dados fica "vigiando" a tabela e o invoca por você!

Para que servem os Triggers?

Triggers resolvem vários problemas cruciais:

- **Auditoria (Logs):** Registrar quem alterou um dado, quando alterou e qual era o valor antigo.
- **Integridade Complexa:** Validar regras de negócio que vão além das simples *Constraints* (`CHECK`, `UNIQUE`).
- **Sincronização de Dados:** Atualizar outras tabelas automaticamente (Ex: ao vender um produto, diminuir o estoque).
- **Cálculos Automáticos:** Derivar valores sempre que uma linha é inserida ou alterada.

Momentos de Disparo

O trigger pode ser configurado para rodar em momentos diferentes em relação ao evento que o acionou:

- **BEFORE (Antes):** Roda antes do dado ser de fato gravado na tabela. Ideal para validações de regras de negócio ou para modificar o dado antes de salvar.
- **AFTER (Depois):** Roda após a gravação. Ideal para atualizar outras tabelas ou criar registros de log/auditoria.
- **INSTEAD OF (Ao invés de):** Executa a ação do trigger no lugar do evento original. Muito usado em *Views* para permitir atualizações complexas.

Eventos de Disparo

Quais ações no banco "puxam" o gatilho? Os Triggers de manipulação de dados (DML) respondem aos comandos:

INSERT	UPDATE	DELETE
Quando uma nova linha é criada.	Quando uma linha existente é modificada.	Quando uma linha é apagada.

Nota: Um mesmo trigger pode ser associado a múltiplos eventos ao mesmo tempo (ex: `BEFORE INSERT OR UPDATE`).

Sintaxe Geral

A sintaxe exata varia muito entre bancos de dados (MySQL, Oracle, SQL Server, PostgreSQL). Em geral:

```
CREATE TRIGGER nome_do_gatilho
{ BEFORE | AFTER } { INSERT | UPDATE | DELETE }
ON nome_da_tabela
FOR EACH ROW -- Executa para cada linha afetada
BEGIN
    -- Lógica SQL a ser executada
END;
```

- **FOR EACH ROW:** Se um `UPDATE` alterar 10 linhas, o gatilho rodará 10 vezes (uma para cada).

Variáveis Especiais: NEW e OLD

Como o Trigger sabe quais dados estão sendo manipulados no momento do disparo? Através de palavras reservadas mágicas!

- **NEW**: Refere-se aos novos dados que estão chegando. (O que você está tentando inserir ou atualizar).
- **OLD**: Refere-se aos dados antigos que já estavam na tabela. (O que você está tentando alterar ou apagar).

Tabela Comparativa: NEW vs OLD

O estado de `NEW` e `OLD` depende do evento!

Evento	OLD	NEW
INSERT	Vazio (`NULL`)	Disponível (Novo dado)
UPDATE	Disponível (Antes da alteração)	Disponível (Após a alteração)
DELETE	Disponível (Dado a ser apagado)	Vazio (`NULL`)

Exemplo Prático 1: Auditoria (Log)

Cenário: Temos uma tabela `funcionarios` e queremos registrar em uma tabela `log_salario` sempre que o salário de alguém for alterado (`UPDATE`).

```
CREATE TABLE funcionarios (
  id INT,
  nome VARCHAR(50),
  salario DECIMAL
);
```

```
CREATE TABLE log_salario (
  func_id INT,
  sal_velho DECIMAL,
  sal_novo DECIMAL,
  data TIMESTAMP
);
```

Exemplo 1: Criando a Função (PostgreSQL)

No **PostgreSQL**, o trigger é dividido em duas partes. Primeiro, criamos uma função que dita a ação:

```
CREATE OR REPLACE FUNCTION log_mudanca_salario()
RETURNS TRIGGER AS $$
BEGIN
    -- Verifica se o salario realmente mudou
    IF NEW.salario <> OLD.salario THEN
        INSERT INTO log_salario (func_id, sal_velho, sal_novo, data)
        VALUES (OLD.id, OLD.salario, NEW.salario, NOW());
    END IF;
    RETURN NEW; -- Retorna a nova linha
END;
$$ LANGUAGE plpgsql;
```

Exemplo 1: Associando o Trigger

Agora, associamos a função criada à tabela usando o comando de Trigger de fato:

```
CREATE TRIGGER trg_audita_salario
AFTER UPDATE ON funcionarios
FOR EACH ROW
EXECUTE FUNCTION log_mudanca_salario();
```

- Usamos `AFTER` porque o log só faz sentido se o update for um sucesso.
- Fica "escutando" `ON funcionarios`.

Exemplo 1: Testando o Gatilho

Ação do Usuário:

```
UPDATE funcionarios SET salario = 5000 WHERE id = 1;
```

Reação do Banco (Invisível para o Usuário): O trigger dispara e injeta uma linha na tabela de logs usando `OLD.salario` (o que estava antes) e `NEW.salario` (os 5000).

Se olharmos a tabela de log depois:

func_id	sal_velho	sal_novo	data
1	4500.00	5000.00	2026-05-22

Exemplo Prático 2: Validação de Regras (BEFORE)

Como impedir que insiram ou atualizem um salário negativo? (Caso Constraints não sejam suficientes).

```
CREATE OR REPLACE FUNCTION valida_salario()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.salario < 0 THEN
        RAISE EXCEPTION 'O salário não pode ser negativo!';
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_valida_salario
BEFORE INSERT OR UPDATE ON funcionarios
FOR EACH ROW EXECUTE FUNCTION valida_salario();
```

Cuidados e Boas Práticas

Cuidado com a Performance

Como rodam silenciosamente `FOR EACH ROW`, se você fizer um `UPDATE` em 1.000.000 de linhas, o trigger roda 1.000.000 de vezes! Isso pode deixar o banco muito lento.

O Efeito Cascata (Loops Infinitos)

Tenha cuidado para que um trigger não dispare outro trigger que dispara o primeiro em um loop infinito. Ex: O trigger A altera a tabela B, que tem um trigger que altera a tabela A... e o banco trava!

Triggers vs Outros Recursos

Quando usar cada coisa?

Recurso	**Uso Ideal**
Constraints	Regras de integridade simples (não pode ser nulo, deve ser único, maior que zero).
Procedure	Quando o usuário deve **escolher** invocar o procedimento manualmente.
Triggers	Quando a ação deve ser **reacionária e inevitável**. Auditoria e validações que ninguém pode burlar.